

7 Relevante regler for logisk algebra

I digital logikk brukes boolsk algebra for å forenkle og analysere logiske uttrykk. I denne rapporten brukes følgende notasjon:

$$A \cdot B = \text{AND}$$

$$A + B = \text{OR}$$

$$\overline{A} = \text{NOT}$$

$$A \oplus B = \text{XOR}$$

Disse reglene er spesielt nyttige når man skal bygge opp eller forenkle logiske porter, for eksempel ved implementering av NOT, NAND, OR, XOR, addere og ALU-logikk.

7.1 Grunnleggende regler

Tabell 27: Grunnleggende regler i boolsk algebra

Navn	Regel	Forklaring
Identitet, AND	$A \cdot 1 = A$	AND med 1 endrer ikke verdien
Identitet, OR	$A + 0 = A$	OR med 0 endrer ikke verdien
Nullregel, AND	$A \cdot 0 = 0$	AND med 0 gir alltid 0
Nullregel, OR	$A + 1 = 1$	OR med 1 gir alltid 1
Idempotent, AND	$A \cdot A = A$	Samme signal AND-et med seg selv
Idempotent, OR	$A + A = A$	Samme signal OR-et med seg selv
Komplement, AND	$A \cdot \overline{A} = 0$	Et signal og det inverterte kan ikke begge være 1
Komplement, OR	$A + \overline{A} = 1$	Enten er signalet eller det inverterte 1
Dobbel invertering	$\overline{\overline{A}} = A$	To inverteringer gir originalt signal

7.2 Kommutative og assosiative regler

Rekkefølgen på inngangene har ikke betydning for AND og OR:

$$A \cdot B = B \cdot A$$

$$A + B = B + A$$

Gruppering av flere ledd har heller ikke betydning for AND og OR:

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

$$(A + B) + C = A + (B + C)$$

Dette betyr at logiske porter med flere innganger kan tegnes og analyseres på flere ekvivalente måter.

7.3 Distributive regler

AND kan distribueres over OR:

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

OR kan også distribueres over AND:

$$A + (B \cdot C) = (A + B)(A + C)$$

Disse reglene brukes ofte når man skal skrive om logiske uttrykk slik at de kan implementeres med bestemte porter.

7.4 Absorpsjonsregler

Absorpsjonsreglene brukes for å fjerne overflødige ledd:

$$A + A \cdot B = A$$

$$A \cdot (A + B) = A$$

Eksempel:

$$A + A \cdot B = A$$

Dette betyr at leddet $A \cdot B$ ikke påvirker resultatet, siden uttrykket allerede blir 1 når $A = 1$.

7.5 De Morgans lover

De Morgans lover er svært viktige når man skal bygge logiske porter ved hjelp av NAND- eller NOR-porter. Lovene viser hvordan invertering av et helt uttrykk kan flyttes inn på hvert enkelt ledd dersom AND byttes med OR, eller OR byttes med AND. Den første De Morgan-loven er:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

Dette betyr at en NAND-funksjon kan tolkes som en OR-funksjon med inverterte innganger. Den andre De Morgan-loven er:

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Dette betyr at en NOR-funksjon kan tolkes som en AND-funksjon med inverterte innganger. Ved å invertere begge sider av uttrykket kan man også skrive OR ved hjelp av NAND:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

Ved bruk av De Morgans lov får vi:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

Dette viser at en OR-port kan bygges ved å invertere begge inngangene og deretter bruke en NAND-port.

7.6 NAND som universell port

En NAND-port er en universell port. Det betyr at man kan bygge alle andre logiske porter kun ved hjelp av NAND-porter. En NOT-port kan bygges ved å koble begge inngangene på en NAND-port sammen:

$$\overline{A} = \overline{A \cdot A}$$

En AND-port kan bygges ved å invertere utgangen fra en NAND-port:

$$A \cdot B = \overline{\overline{A \cdot B}}$$

En OR-port kan bygges ved hjelp av De Morgans lov:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

Dermed kan NAND brukes som byggestein for NOT, AND, OR og mer komplekse funksjoner.

7.7 XOR-regler

XOR står for Exclusive OR, og gir 1 bare når inngangene er forskjellige. XOR kan uttrykkes som:

$$A \oplus B = \overline{A}B + A\overline{B}$$

Sannhetstabellen for XOR er:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Noen nyttige XOR-regler er:

$$A \oplus 0 = A$$

$$A \oplus 1 = \overline{A}$$

$$A \oplus A = 0$$

$$A \oplus \overline{A} = 1$$

Disse reglene er spesielt viktige i en adder/subtraktor-krets. Ved å XOR-e et bit med kontrollsignalet SUB , kan man velge om bitet skal inverteres eller ikke:

$$B' = B \oplus SUB$$

Når $SUB = 0$ får vi:

$$B' = B \oplus 0 = B$$

Når $SUB = 1$ får vi:

$$B' = B \oplus 1 = \overline{B}$$

Dette gjør XOR-porten nyttig for å velge mellom addisjon og subtraksjon i en ALU.